



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: 实验 5: 散列查找

专 业 班 级 通信工程 三班

学 生 学 号 _____

学 生 姓 名 _____

实 验 时 间 2025 年 5 月 20 日

实验报告

【实验目的】

掌握散列查找的基本概念及其基本操作的实现；

【实验原理】

为避免像数组查找数据那样糟糕的效率，我们将每个数据元素计算哈希值（使用除留余数法，因此 N （哈希表大小）最好为一个素数，使得哈希值重复），将其存放在哈希表中。同样的，为避免重复填入哈希值，我们需要一个状态值以表示对应空格是否被填充，于是哈希表的定义如下：

```
typedef struct HashTable {  
    int *data;        // 存储数据的数组  
    int *status;      // 槽位状态数组：0-空 1-已用 2-已删除  
    int capacity;     // 哈希表总容量（始终保持为质数）  
    int size;         // 当前存储元素数量  
} HashTable;
```

为了处理哈希值冲突的情况，我们有线性探测法来插入冲突的元素。其原理为以线性序列循环探测下一个可用的槽位。实际实现为：

- (1) 所插元素是否已存在
- (2) For 循环查找可用位置，优先选择状态为已删除的元素位置，不然则插入到空位置。

查找哈希表中的数据同样依照线性探测法，在所得哈希值位置向后探测查找，如果遇到空位置，说明该元素没有插入哈希表。

- (1) 计算哈希值，循环查找其后元素
- (2) 遇到空位置，停止遍历

【实验内容与要求】

内容：

- 1) 创建一个hash 空表，实现动态空间分配的初始化;
- 2) 构造一个hash 函数和某种处理冲突的方法将数据元素存放到hash 表中;
- 3) 设计相应的hash 查找函数对hash 表中的元素进行查找;
- 4) hash 构造函数可选用除留余数法或其他方法;
- 5) 处理冲突可选开放定址法或其他方法。

要求：

- 1) 构造哈希函数，实现哈希查找。

【实验源程序】

1.Hash 头文件

```
#ifndef _Hash_H
#define _Hash_H

// 哈希表结构定义
typedef struct HashTable {
    int *data;        // 存储数据的数组
    int *status;      // 槽位状态数组：0-空 1-已用 2-已删除
    int capacity;     // 哈希表总容量（始终保持为质数）
    int size;         // 当前存储元素数量
} HashTable;

int hash(int key, int capacity);
HashTable* createHashTable(int initialCapacity);
void resize(HashTable* table, int newCapacity);
int insert(HashTable* table, int key);
int search(HashTable* table, int key, int* pos);
void printHashTable(HashTable* table);

#endif
```

2.bitree 函数文件

```
#include<stdio.H>
```

```

#include "Hash.h"
#include<stdlib.h>

/* 判断是否为质数的函数
 * 参数: n - 要判断的数字
 * 返回值: true-是质数, false-不是质数
 */
int isPrime(int n) {
    if (n <= 1) return 0;
    if (n <= 3) return 1; // 2 和 3 是质数
    // 排除能被 2 或 3 整除的数
    if (n % 2 == 0 || n % 3 == 0) return 0;
    // 检查 6k±1 形式的因数
    for (int i = 5; i * i <= n; i += 6)
        if (n % i == 0 || n % (i + 2) == 0) return 0;
    return 1;
}

/* 获取下一个质数的函数
 * 参数: n - 起始数字
 * 返回值: 大于 n 的最小质数
 */
int nextPrime(int n) {
    if (n <= 1) return 2;
    while (!isPrime(++n)); // 循环查找直到找到质数
    return n;
}

/* 哈希函数 (除留余数法)
 * 参数: key - 键值, capacity - 当前哈希表容量
 * 返回值: 计算得到的哈希值
 */
int hash(int key, int capacity) {
    return key % capacity; // 对容量取模
}

/* 创建哈希表
 * 参数: initialCapacity - 初始容量
 * 返回值: 初始化好的哈希表指针
 */
HashTable* createHashTable(int initialCapacity) {
    HashTable* table = (HashTable*)malloc(sizeof(HashTable));
    // 将容量设置为比初始容量大的最小质数
    table->capacity = nextPrime(initialCapacity);
}
    
```

```

        table->size = 0;
        // 分配存储空间
        table->data = (int*)malloc(table->capacity * sizeof(int));
        table->status = (int*)malloc(table->capacity * sizeof(int));
        // 初始化所有槽位状态为空
        for (int i = 0; i < table->capacity; i++)
            table->status[i] = 0;
        return table;
    }

/* 再散列
 * 参数: table - 哈希表指针, newCapacity - 新容量
 */
void resize(HashTable* table, int newCapacity) {
    // 创建新数组
    int* newData = (int*)malloc(newCapacity * sizeof(int));
    int* newStatus = (int*)malloc(newCapacity * sizeof(int));
    for (int i = 0; i < newCapacity; i++)
        newStatus[i] = 0; // 初始化新状态数组

    // 保存旧数据指针
    int oldCapacity = table->capacity;
    int* oldData = table->data;
    int* oldStatus = table->status;

    // 更新表属性
    table->capacity = newCapacity;
    table->data = newData;
    table->status = newStatus;
    table->size = 0; // 重置大小, 插入时会重新计数

    // 重新插入所有有效元素
    for (int i = 0; i < oldCapacity; i++) {
        if (oldStatus[i] == 1) { // 只处理有效元素
            int key = oldData[i];
            int idx = hash(key, newCapacity);
            // 处理新表中的冲突
            while (table->status[idx] == 1)
                idx = (idx + 1) % newCapacity;
            table->data[idx] = key;
            table->status[idx] = 1;
            table->size++;
        }
    }
}

```

```

        // 释放旧空间
        free(oldData);
        free(oldStatus);
    }

    /* 插入元素到哈希表
     * 参数: table - 哈希表指针, key - 要插入的键值
     * 返回值: true-插入成功, false-键已存在
     */
    int insert(HashTable* table, int key) {
        // 检查负载因子, 超过 0.7 时扩容
        if (table->size * 10 >= table->capacity * 7) {
            resize(table, nextPrime(table->capacity * 2));
        }

        int idx = hash(key, table->capacity);
        int firstDeleted = -1; // 记录遇到的第一个已删除位置

        // 线性探测查找可用槽位
        for (int i = 0; i < table->capacity; i++) {
            int current = (idx + i) % table->capacity;

            if (table->status[current] == 1) { // 槽位已用
                if (table->data[current] == key)
                    return 0; // 键已存在
            }
            else if (table->status[current] == 2) { // 槽位已删除
                if (firstDeleted == -1) // 记录第一个可复用的位置
                    firstDeleted = current;
            }
            else { // 找到空槽位
                if (firstDeleted != -1) // 优先使用已删除的位置
                    current = firstDeleted;
                table->data[current] = key;
                table->status[current] = 1;
                table->size++;
                return 1;
            }
        }
    }
}

```

```

/* 查找元素
 * 参数: table - 哈希表指针, key - 查找键值, pos - 用于返回位置
 * 返回值: true-找到, false-未找到
 */
int search(HashTable* table, int key, int* pos) {
    int idx = hash(key, table->capacity);
    for (int i = 0; i < table->capacity; i++) {
        int current = (idx + i) % table->capacity;
        if (table->status[current] == 0) { // 遇到空槽位停止查找
            *pos = -1;
            return 0;
        }
        if (table->status[current] == 1 && table->data[current] == key)
        {
            *pos = current;
            return 1;
        }
    }
    *pos = -1;
    return 0;
}

/* 打印哈希表
 * 参数: table - 哈希表指针
 */
void printHashTable(HashTable* table) {
    printf("Capacity: %d\nSize: %d\n", table->capacity, table->size);
    for (int i = 0; i < table->capacity; i++) {
        printf("Index %d: ", i);
        if (table->status[i] == 1)
            printf("%d\n", table->data[i]);
        else
            printf("%s\n", table->status[i] == 2 ? "Deleted" : "Empty");
    }
}
    
```

3.主程序应用示例

```

#include<stdio.h>
#include<stdlib.h>
#include "Hash.h"
    
```

```
int main() {
    // 创建初始容量为 11 的哈希表
    HashTable* table = createHashTable(11);

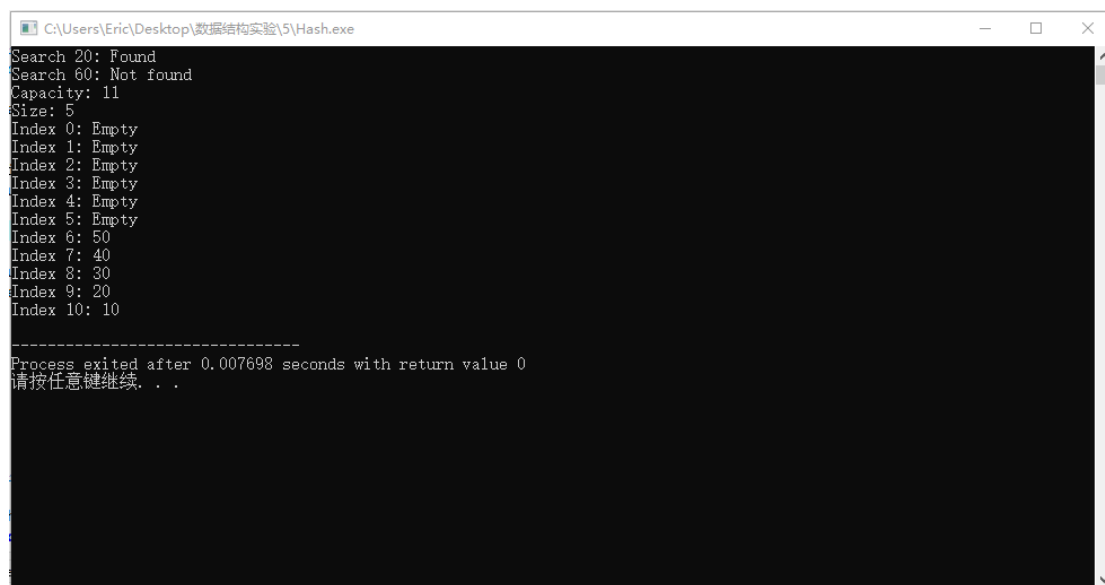
    // 插入测试数据
    insert(table, 10);
    insert(table, 20);
    insert(table, 30);
    insert(table, 40);
    insert(table, 50);

    // 查找测试
    int position;
    printf("Search 20: %s\n", search(table, 20, &position) ? "Found" :
    "Not found");
    printf("Search 60: %s\n", search(table, 60, &position) ? "Found" :
    "Not found");

    // 打印哈希表当前状态
    printHashTable(table);

    // 释放内存
    free(table->data);
    free(table->status);
    free(table);
    return 0;
}
```

【实验结果】



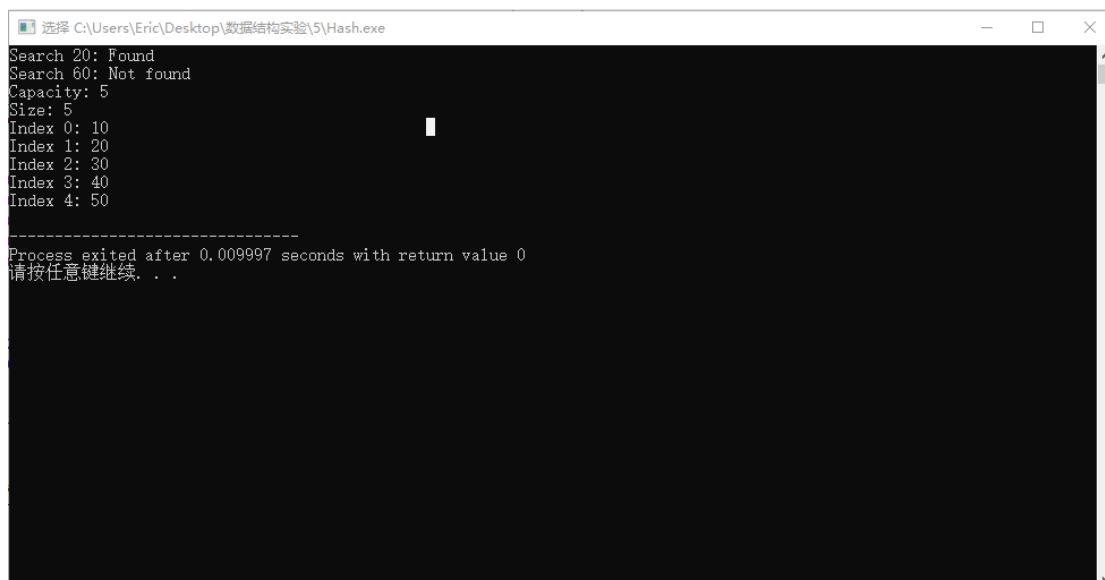
```

C:\Users\Eric\Desktop\数据结构实验\5\Hash.exe
Search 20: Found
Search 60: Not found
Capacity: 11
Size: 5
Index 0: Empty
Index 1: Empty
Index 2: Empty
Index 3: Empty
Index 4: Empty
Index 5: Empty
Index 6: 50
Index 7: 40
Index 8: 30
Index 9: 20
Index 10: 10

-----
Process exited after 0.007698 seconds with return value 0
请按任意键继续. . .
    
```

图 1 运行结果截图

我们输入的数据为 10, 20, 30, 40, 50, 且初始设置哈希表的大小为 11, 此时, 所有元素不发生冲突且成功插入与查找元素。



```

选择 C:\Users\Eric\Desktop\数据结构实验\5\Hash.exe
Search 20: Found
Search 60: Not found
Capacity: 5
Size: 5
Index 0: 10
Index 1: 20
Index 2: 30
Index 3: 40
Index 4: 50

-----
Process exited after 0.009997 seconds with return value 0
请按任意键继续. . .
    
```

图 2 初始容量为 5

当初始设置哈希表的大小为 5, 此时, 所有元素均发生冲突但仍然成功插入与查找元素。说明线性探测法有效。

【实验心得】

本次实验，我们实现了散列查找的相关函数与解决哈希值冲突的一系列操作，途中遇到了许多困难，比如在实现哈希表插入函数时，我忘记处理数据重复的情况，在我输入多个重复数据后，多个重复数据出现在表中，这是与哈希表的定义不符的。在此次实验的一些基础上，我也想到了一些可以改进的方面并加以改进：

在实际运用，我们往往会需要动态调整表的大小，同时，由课上所学，装载因子在超过 0.7 后，哈希值冲突的可能性大大提升，于是我们将以此为条件，在插入数据时随时进行再散列，这将大大提升表的性能：

- (1) 装载因子没有超过 0.7，正常插入
- (2) 超过 0.7，重新创建一个新的哈希表，大小为当下容量 2 倍的下一个素数
- (3) 插入已有的元素

实现代码如下：

```
/* 再散列
 * 参数: table - 哈希表指针, newCapacity - 新容量
 */
void resize(HashTable* table, int newCapacity) {
    // 创建新数组
    int* newData = (int*)malloc(newCapacity * sizeof(int));
    int* newStatus = (int*)malloc(newCapacity * sizeof(int));
    for (int i = 0; i < newCapacity; i++)
        newStatus[i] = 0; // 初始化新状态数组

    // 保存旧数据指针
    int oldCapacity = table->capacity;
    int* oldData = table->data;
    int* oldStatus = table->status;

    // 更新表属性
    table->capacity = newCapacity;
    table->data = newData;
    table->status = newStatus;
    table->size = 0; // 重置大小，插入时会重新计数

    // 重新插入所有有效元素
    for (int i = 0; i < oldCapacity; i++) {
        if (oldStatus[i] == 1) { // 只处理有效元素
            int key = oldData[i];
            int idx = hash(key, newCapacity);
```

```

        // 处理新表中的冲突
        while (table->status[idx] == 1)
            idx = (idx + 1) % newCapacity;
        table->data[idx] = key;
        table->status[idx] = 1;
        table->size++;
    }
}

// 释放旧空间
free(oldData);
free(oldStatus);
}

/* 插入元素到哈希表
 * 参数: table - 哈希表指针, key - 要插入的键值
 * 返回值: true-插入成功, false-键已存在
 */
int insert(HashTable* table, int key) {
    // 检查负载因子, 超过 0.7 时扩容
    if (table->size * 10 >= table->capacity * 7) {
        resize(table, nextPrime(table->capacity * 2));
    }

    int idx = hash(key, table->capacity);
    int firstDeleted = -1; // 记录遇到的第一个已删除位置

    // 线性探测查找可用槽位
    for (int i = 0; i < table->capacity; i++) {
        int current = (idx + i) % table->capacity;

        if (table->status[current] == 1) { // 槽位已用
            if (table->data[current] == key)
                return 0; // 键已存在
        }
        else if (table->status[current] == 2) { // 槽位已删除
            if (firstDeleted == -1) // 记录第一个可复用的位置
                firstDeleted = current;
        }
        else { // 找到空槽位
            if (firstDeleted != -1) // 优先使用已删除的位置
                current = firstDeleted;
            table->data[current] = key;
            table->status[current] = 1;
            table->size++;
        }
    }
}

```

```

        return 1;
    }
}
}

```

重新运行：

The screenshot shows a C program being compiled and executed. The code in `main.c` defines a hash table with 11 slots and inserts the values 10, 20, 30, 40, and 50. It then searches for 20 and 60, prints the table state, and frees the memory. The output window `Hash.exe` shows the search results: 'Search 20: Found' and 'Search 60: Not found', followed by the table state and a message indicating the process exited after 0.008429 seconds.

```

1 #include<stdio.h>
2 #include<stdlib.h>
3 #include "Hash.h"
4
5
6
7 int main() {
8     // 创建初始容量为11的哈希表
9     HashTable* table = createHashTable(5);
10
11     // 插入测试数据
12     insert(table, 10);
13     insert(table, 20);
14     insert(table, 30);
15     insert(table, 40);
16     insert(table, 50);
17
18     // 查找测试
19     int position;
20     printf("Search 20: %s\n", search(table, 20, &position));
21     printf("Search 60: %s\n", search(table, 60, &position));
22
23     // 打印哈希表当前状态
24     printHashTable(table);
25
26     // 释放内存
27     free(table->data);
28     free(table->status);
29     free(table);

```

```

Search 20: Found
Search 60: Not found
Capacity: 11
Size: 5
Index 0: Empty
Index 1: Empty
Index 2: Empty
Index 3: Empty
Index 4: Empty
Index 5: Empty
Index 6: 50
Index 7: 40
Index 8: 30
Index 9: 20
Index 10: 10
-----
Process exited after 0.008429 seconds with return value 0
请按任意键继续. . .

```

图 3 改进结果